Mock Final Exam - Version V2

Total Points: 100

Mock Final Exam - Version 2

Total Points: 100

Multiple Choice (12 questions, 3 points each = 36 points)

Select the best answer for each question.

1. (3 points) Which stream allows you to read primitive data types (like int, double, boolean) directly from a binary file, interpreting the raw bytes correctly?

A. FileInputStream

B. FileReader

C. ObjectInputStream

D. DataInputStream

2. (3 points) What is the purpose of the transient keyword in the context of object serialization?

A. To mark a method that should not be called during serialization.

B. To indicate that a class cannot be serialized at all.

C. To prevent a specific instance variable from being saved during the serialization of an object.

D. To ensure an instance variable is always included during serialization.

3. (3 points) If you open a FileOutputStream for an existing file without specifying the append mode (or setting it to false), what happens to the original file content?

A. New data is appended to the end of the existing content.

B. An IOException is thrown because the file already exists.

C. The original content is overwritten by the new data.

D. A backup of the original file is created automatically.

4. (3 points) When reading data from a DataInputStream, what typically happens if you try to read past the end of the file?

A. The read() method returns null.

B. An EOFException is thrown.

C. The stream automatically loops back to the beginning.

D. A -1 value is returned consistently for all read methods.

5. (3 points) A recursive method typically has two main parts:

A. Input and Output

B. Loop and Condition

C. Base Case and Recursive Step

D. Try and Catch

6. (3 points) Consider the recursive definition: power(base,exp)=1 if exp==0, and power(base,exp)=base*power(base,exp-1) if exp>0. How many recursive calls are made to calculate power(2,3)?

A. 1

B. 2

C. 3

D. 4

7. (3 points) What is tail recursion?

A. A recursion where the base case is at the end of the method.

B. A recursion that uses a helper method.

C. A recursion where the recursive call is the very last operation in the method, with no pending computations.

D. A recursion that operates on the tail end of a data structure like a list.

8. (3 points) In JavaFX, which class is the base class for all visual elements like shapes, controls, and layout panes?

A. Scene

B. Stage

C. Node

D. Application

9. (3 points) What event type would you typically listen for to detect when the user presses and releases a keyboard key while a specific JavaFX control has focus?

A. MouseEvent

B. ActionEvent

C. KeyEvent

D. WindowEvent

10. (3 points) What is the primary role of an interface in Java?

A. To provide concrete implementations for subclasses.

B. To define a contract of methods that implementing classes must provide.

C. To allow multiple inheritance of instance variables.

D. To group static utility methods.

11. (3 points) If class Dog extends abstract class Animal, and Animal has an abstract method makeSound(), what must the Dog class do?

A. It must declare makeSound() as abstract as well.

B. It must provide a concrete implementation for makeSound().

C. It can optionally implement makeSound().

D. It cannot extend Animal if Animal has abstract methods.

12. (3 points) Which method from the Object class is often overridden in conjunction with equals() to ensure consistency, especially when objects are stored in hash-based collections?

A. toString()

B. clone()

C. hashCode()

D. finalize()

Short Answer / Code Reading

1. (4 questions, variable points = 29 points)

2. (6 points) Explain why buffering I/O operations (using classes like BufferedInputStream or BufferedOutputStream) can significantly improve performance, especially when dealing with large files.

3. (7 points) Consider this recursive Java method:

```
public static void countDown(int num) {
if (num <= 0) {
System.out.println("Blast off!");
} else {
System.out.print(num + "...");
countDown(num - 1);
```

```
}
}
```

What exact output will be printed to the console when countDown(3) is called?

4. (8 points) What is the difference between an abstract class and an interface in Java? List at least two key differences regarding constructors, instance variables, method implementations, and inheritance.

5. (8 points) You have a binary file readings.dat where each record consists of a long timestamp followed by an int sensor ID and then a float reading value. Explain how you would read all records from this file using DataInputStream until the end of the file is reached. Describe the loop structure and the exception handling involved.

Coding / Program Design

1. (4 questions, variable points = 35 points)

2. (10 points) Write a recursive Java method countOccurrences(String str, char target) that counts and returns the number of times the character target appears in the string str. Do not use any built-in string methods for counting or searching (like indexOf, contains, etc., though you will need methods like length(), charAt(), and substring()). Define the base case and recursive step clearly within your code using comments.

```
public static int countOccurrences(String str, char target) {
// Base Case(s): ??
// Recursive Step: ??
```

}

3. (12 points) Create a Java class TemperatureReading implementing Serializable. It should store location (String), timestamp (long - milliseconds since epoch), and celsiusTemp (double). Include a constructor and necessary accessors. Then, write a main method snippet that:

- Reads existing TemperatureReading objects from a file temps_archive.dat using ObjectInputStream (assume the file contains one or more serialized TemperatureReading objects).
- Prints the location and temperature of each reading to the console.
- Handles potential IOException, ClassNotFoundException, and EOFException.

4. (13 points) Write a recursive Java method reversePrint(LinkedList<String> list) that prints the elements of a LinkedList of strings in reverse order. You should use recursion and the list's methods like isEmpty(), getFirst(), removeFirst() (or similar) to achieve this. The method should modify the list as it processes it (it's okay if the list is empty after the call). Hint: Consider what needs to happen before and after the recursive call to achieve reverse order.

```
import java.util.LinkedList;

public static void reversePrint(LinkedList<String> list) {
// Base Case: ??
// Recursive Step: ?? (Think about print timing)
}
```